

Algo 框架

目录

一、 概述	4
什么是 Algo 框架?	4
Algo 对象	5
DataSet 接口简介	6
DataSet 引用关系树	8
RowMeta、Field 和 DataType 简介	9
数据类型	9
二、 常用 API	10
获取 DataSet 的常用方式	10
DataSet 接口方法	12
select	14
addField	15
addNullField	16
addBalanceField	17
removeField	18
where/filter	18
orderBy	20
groupBy	21
union	22
join	23
leftjoin	26

count	26
print	27
copy	28
三、 高级 API	28
cache	28
Cache Builder	30
DataSetBuilder	30
Input	31
计算表达式	32
表达式语法	32
日期函数	34
字符串函数	35
聚合函数	35
特殊函数	37
四、 注意事项	37
关闭 DataSet	37
其他注意事项	38
ORM 中的 QFilter 是否可以直接在 Algo 中使用?	39
五、 案例演示	39
六、 操作实践	43

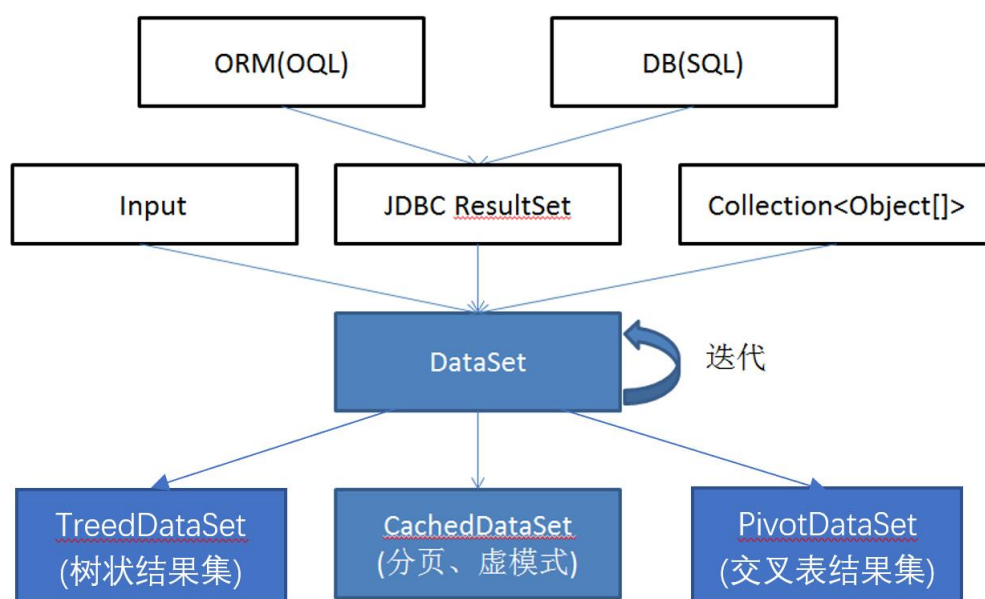
一、概述

什么是 Algo 框架?

Algo 是内存数据库计算引擎，提供 SQL 计算功能，支持常用的 SQL 算法，遵循 SQL92 标准。

Algo 是一个迭代计算框架。

Algo 提供内存计算能力，能实现类似数据库的各种操作，但是它本身不是数据库，不负责存储数据。它并不能进行分布式计算，是单 Java 进程节点内部运行。它也不依赖于关系数据库运行。



应用场景

- 需在内存中对数据进行过滤、聚合、分组、join 等各种复杂内存计算的场景。
- Algo 通过宿主机内存计算代替数据库计算，减轻数据库压力，解决数据库性能瓶颈，适用于大数据量且需要进行多次计算的场景。

- 数据库分库后，替代 SQL 功能
- 替代复杂 SQL
- 替代临时表
- 作为报表开发框架

Algo 对象

接口/对象类	用途说明
kd.bos.algo.Algo	创建 DataSet getCacheDataSet(String cacheld) 这个类是入口，给平台使用的，业务不要使用
kd.bos.algo.DataSet	DataSet 操作接口，包括转换，action，close 等
Kd.bos.algo.RowMeta	DataSet 的元数据，类似 jdbc ResultSetMetaData 由字段 Field[] 组成
Kd.bos.algo.Field	字段
Kd.bos.algo.DataType	数据类型，包括主要数据类型
Kd.bos.algo.Input	数据源接口
Kd.bos.algo.Row	行接口，行值访问器，Row 是一个虚接口，访问器，不是 Object[]，所以业务代码不能把 Row 缓存起来
CachedDataSet	DataSet.cache 的结果对象，不是 DataSet

	的子类
CacheHint	Cache hint, 指示
JoinDataSet	DataSet.join 的结果对象,也是 Join 构造器, 不是 DataSet 的子类
HashJoinDataSet	DataSet.hahJoin 的结果对象, 也是 HashJoin 构造器, 不是 DataSet 的子类
GroupbyDataSet	DataSet.groupBy 的结果对象, 也是 groupBy 构造器, 不是 DataSet 的子类
HashTable	DataSet.toHashTable 的结果对象, 是 hashJoin 的参数
JoinType	Join 类型枚举
JoinHint	Join hint, 指示
RowMetaFactory	创建 RowMeta, 给平台用的

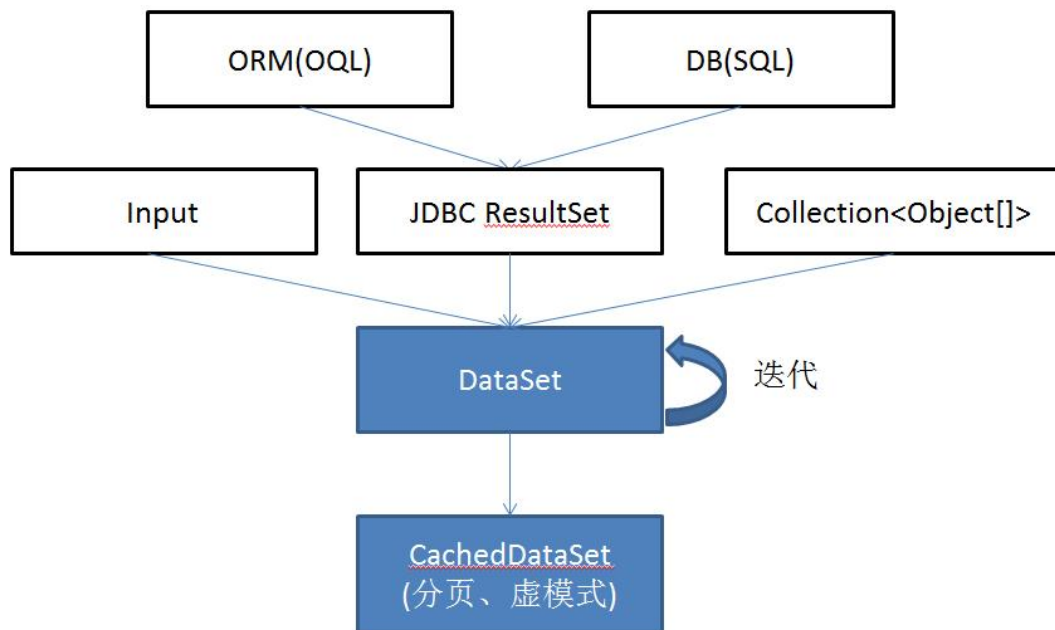
DataSet 接口简介

DataSet 是指 `kd.bos.algo.DataSet`, 无特殊说明情况下, 后文中的 DataSet 都是指 `kd.bos.algo.DataSet`, DataSet 是结果集, 与 JDBC 中的 ResultSet 类似。

DataSet 即是结果集转换器, 也是迭代计算。

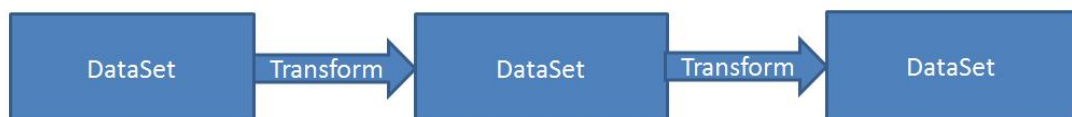
DataSet 不局限于数据库中的数据, 文件流, 集合类型的数据也可以转换为 DataSet。

获取的 DataSet 也可以缓存。

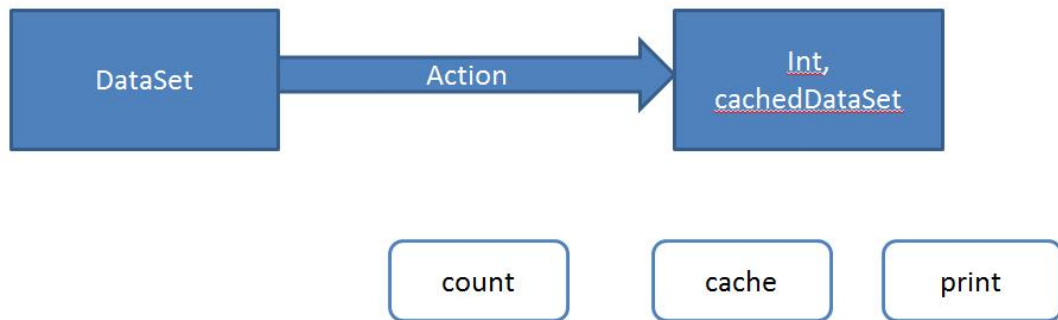


kd.bos.algo.DataSet 是两类接口，Transform 和 Action，相关概念与 Apache Flink 是对等的。

Transform：转换 DataSet，生成新的 DataSet；包括主要 sql 操作，如 orderby，select，where，filter，group by，range，top，union 等，可以执行 sql 语句，executeSql。



Action：Action 是消费 DataSet，Action 执行结果不是 DataSet；Action 执行后，DataSet 自动关闭；Action 接口不多，比如 count，cache 等。



DataSet 引用关系树

DataSet 经过多次转换后，会形成一个引用关系树，如：

```
DataSet c=a.union(b);
```

```
DataSet e=c.select(new String[]{"fid","fname"});
```

```
DataSet f=c.join(d,xxx).finish();
```

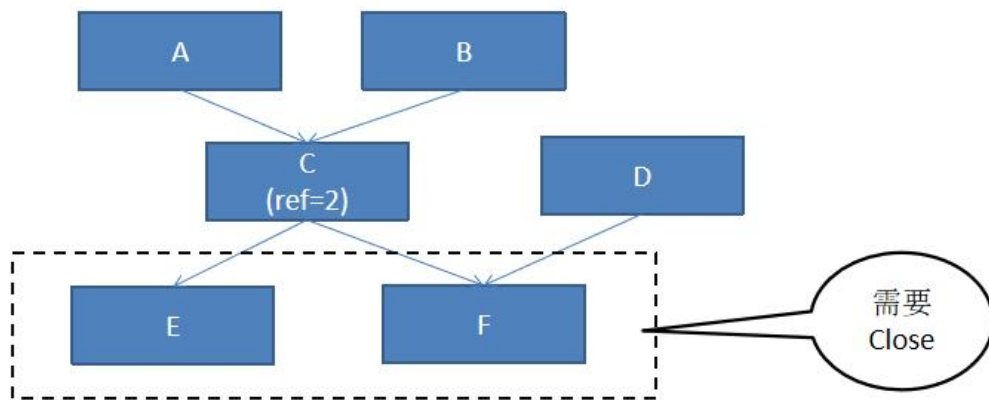
非叶子节点都是被引用的 DataSet

每个 DataSet 节点都会消耗资源或者引用资源，close 时会释放资源

DataSet.close 时，会自动 close 引用节点

所有叶子节点 close，会自动 close 整棵树

叶子节点必须 close



RowMeta、Field 和 DataType 简介

RowMeta、Field 和 DataType 对应数据库中的行数据、字段和数据类型。

获取 RowMeta

```
RowMeta rowMeta = dataSet.getRowMeta();
```

获取 Field

```
Field field = rowMeta.getField(index);
```

获取 DataType

```
DataType type = field.getDataType();
```

数据类型

类型	说明
BooleanType	boolean 类型
StringType	字符串类型
IntegerType	整数类型

LongType	长整数类型
DoubleType	从元数据 ORM 框架查出来的数据不会有 Double
BigDecimalType	BigDecimal 类型
DateType	日期类型
TimestampType	Timestamp 类型
NullType	Select 中动态创建 null Field 生成
UnknownType	表达式解析后无法确定的列，业务不会用到

二、常用 API

获取 DataSet 的常用方式

Algo 相关接口定义和实现存在于 bos-algo-1.0.jar 中。在实际使用过程中主要使用

DataSet 的相关接口方法。

方式一：Algo 原生创建

```
// algo 原生创建
String algoKey = this.getClass().getName();
Algo algo = Algo.create(algoKey);
Collection<Object[]> collection = new ArrayList<Object[]>();
Object[] obj1 = new Object[] { "1", "唐僧", "男", "23" };
Object[] obj2 = new Object[] { "2", "悟空", "女", "24" };
Object[] obj3 = new Object[] { "3", "八戒", "男", "25" };
Object[] obj4 = new Object[] { "4", "沙僧", "女", "26" };
collection.add(obj1);
collection.add(obj2);
collection.add(obj3);
collection.add(obj4);
RowMeta rm = new RowMeta(new String[] { "id", "name", "sex", "age" }, new
DataType[] { DataType.IntegerType,DataType.StringType, DataType.StringType,
DataType.IntegerType });
DataSet ds = algo.createDataSet(collection, rm);
```

业务使用方式

方式一： QueryServiceHelper.queryDataSet(String algoKey, String

entityName,String selectFields, QFilter[] filters,String orderBys)

```
// 方式一： QueryServiceHelper.queryDataSet(String algoKey, String entityName,String
selectFields, QFilter[] filters,String orderBys)
String algoKey = this.getClass().getName();
String entityName = "qyrv_triptype";
String selectFields = "number,name,status";
QFilter[] filters = new QFilter[] { new QFilter("status", QCP.equals, "A") };
String orderBys = "number desc";
DataSet ds = QueryServiceHelper.queryDataSet(algoKey, entityName,selectFields, filters,
orderBys);
System.out.println(ds);
```

方式二： ORM.create().queryDataSet(String algoKey, String entityName, String

selectFields, QFilter[] filters, String orderBys)

```
// 方式二: ORM.create().queryDataSet(String algoKey, String entityName, String
// selectFields, QFilter[] filters, String orderBys)
String algoKey = this.getClass().getName();
String entityName = "qyrv_triptype";
String selectFields = "number,name,status";
QFilter[] filters = new QFilter[] { new QFilter("status", QCP.equals, "A") };
String orderBys = "number desc";
DataSet ds2 = ORM.create().queryDataSet(algoKey, entityName, selectFields,
filters, orderBys);
ds2.print(true);
```

方式三:DB.queryDataSet();

```
//方式三:DB.queryDataSet();
String algoKey = this.getClass().getName();
String sql = "select fid,fnumber,fname,fstatus from tk_qyrv_triptype where fstatus=?";
DataSet ds3 = DB.queryDataSet(algoKey, DBRoute.base, sql, new Object[] { "A" });
ds3.print(true);
```

DataSet 接口方法

接口方法	说明
print	打印输出到 console
close	关闭
select	选择字段。类似:select a,b,c, a,b,c 可以是表达式
addField	增加字段
addNullField	增加 null 字段, 等价于 addField("null" ,alias)
addBalanceField	增加余额字段, 该字段的值从上往下累加
removeField	删除字段

filter	过滤, 类似 where, expr 中可以写变量, 变量值通过 params 传递, 比如 "a>b", params 是 map, key 包含 a 和 b
where	过滤
orderBy	排序, field 可以写成 field asc, field desc
groupBy	groupBy 操作
executeSql	执行 sql
join	join 操作
leftJoin	leftJoin 操作
rightJoin	rightJoin 操作
fullJoin	fullJoin 操作
hashJoin	hashJoin 操作
union	union 操作
top	取 top n 个
limit	取部分行
range	取部分行, 类似 limit
copy	拷贝自己
count	对 field 进行 count 计数, distinct 消除重复
cache	将 dataset 缓存起来
cacheBuilder	

select

选择字段。类似:select a,b,c, a,b,c 可以是表达式

```
public DataSet select(String[] exprs)
```

```
public DataSet select(String exprs);
```

参数	类型	说明
exprs	String[]	exprs 可以是字段 field、Field as alias、表达式 as alias;Select是对源 DataSet 的重新选择，类似 sql; DataSetA.Select(new String[]{ "a" ," b+c as d" }) 等价于 Select a,b+c as d from DataSetA。
exprs	String	exprs 可以是直接写多个字段，比如 dataSetA.select("fname, famount as amount,1+2 as c");

返回选择字段后的 DataSet

select 示例代码

```

//select 方法示例
DataSet ds= dataSet.select(new String[]{
    "famount",
    "famount c1",
    "famount + fqty c2",
    "famount * fqty c3",
    "null c4", "true c5",
    "(famount + fqty) * 3 c6",
    "famount >= fqty c7",
    "famount >= fqty and fqty<100 c8",
    "fname + 'abc' name"});

// select 方法
DataSet ds1 = ds3.select("fnumber,fname");
    while (ds1.hasNext()) {
        Row r = ds1.next();
        System.out.println(r.get(0) + ":" + r.get(1));
    }

//select 方法
DataSet ds2 = ds3.select(new String[] { "fnumber", "fname" });
DataSet ds22 = ds2.copy();
while (ds2.hasNext()) {
    Row r = ds2.next();
    System.out.println(r.get(0) + ":" + r.get(1));
}

```

addField

添加字段。

// 在现有 DataSet 基础上添加字段，或者删除字段

public DataSet addField(String expr, String alias)

// 添加表达式 expr 字段，数据类型自动判断

public DataSet addField(String expr, String alias)

参数	类型	说明
----	----	----

expr	String	expr 可以是字段 field、Field as alias、表达式 as alias
alias	String	字段别名

返回添加完字段后的 DataSet。

```
//addField
DataSet ds222 = ds22.addField("1+1", "status");
ds222.print(true);
```

addNullField

添加值为 null 的字段，数据类型为 NullType。

public DataSet addNullField(String alias)

public DataSet addNullField(String... aliases)

```
// addNullField
DataSet ds33 = ds22.addNullField(s: "tel");
while (ds33.hasNext()) {
    Row r = ds33.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}

DataSet ds3 = ds22.addNullField(new String[] { "tel", "QQ" });
while (ds3.hasNext()) {
    Row r = ds3.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2) + ":" +
        r.get(3));
}
```


参数	类型	说明
alias	String	别名
aliases	String...	别名数组

返回添加完字段后的 DataSet。

addBalanceField

添加余额字段，addBalanceField 方法将字段的值从上往下累加

```
public DataSet addBalanceField(String expr, String alias)
```

参数	类型	说明
expr	String	表达式（值）
alias	String	别名

返回添加完字段后的 DataSet。

```
addBalanceField("amount", "balance")
```

```
addBalanceField("amount * 2", "balance2")
```

addBalanceField 案例

Period	Amount	balance	balance2
201701	100	100	200
201702	200	300	600
201703	200	500	1000

```
//addBalanceField("值", "balance")
DataSet ds4 = ds22.addBalanceField(s: "100", s1: "balance");
//ds3 = ds3.addBalanceField("amount*2", "balance2");
while (ds4.hasNext()) {
    Row r = ds4.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

removeField

删除字段。

public DataSet removeField(String field)

参数	类型	说明
field	String	字段名称

返回删除完字段后的 DataSet

```
// 删除字段 removeField(String field)
DataSet ds1 = ds3.removeFields(...strings: "fname");
while (ds1.hasNext()) {
    Row r = ds1.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

where/filter

过滤，类似 where，expr 中可以写变量，变量值通过 params 传递，比如 "a>b"，params 是 map，key 包含 a 和 b。filter 类似于 sql 的 where 条件。where 和 filter 是完全等价的

public DataSet where(String expr)

```
public DataSet where(String expr, Map<String, Object> params)
```

```
public DataSet filter(String expr)
```

```
public DataSet filter(String expr, Map<String, Object> params)
```

参数	类型	说明
expr	String	expr 是计算表达式，必须为 boolean
params	Map<String, Object>	params 传递参数给 expr。 如 expr 为 famount + var1 > 10000。params 为： {var1: 100}等价于： famount+100>10000。

where 示例

```
// where/filter
DataSet ds1 = ds.where( expr: "sex = '男'");
while (ds1.hasNext()) {
    Row r = ds1.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}

Map<String, Object> params = new HashMap<String, Object>();
params.put("sex1", "女");
ds1 = ds.where( expr: "sex =sex1", params);
while (ds1.hasNext()) {
    Row r = ds1.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

orderBy

排序

public DataSet orderBy(String[] exprs)

参数	类型	说明
expr	String[]	expr 可以是:字段 Fname, 默认为 asc 排序。Fname asc、Fname desc、Expr 不可以为表达式

```
// orderBy
DataSet ds1 = ds.orderBy(new String[] { "age desc" });
while (ds1.hasNext()) {
    Row r = ds1.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

groupBy

groupBy 操作与 SQL 中的类似

1.GroupBy 结果集默认根据 group fields 排序，默认 asc 排序，可以对每个 field 的排序方向进行设置，即以下 orderByDescs:

GroupbyDataSet groupBy(String[] groupFields, boolean[] orderByDescs);

所以，对 groupby 结果进行 orderBy，请考虑是否有必要。

2. SQL 92 标准，group by 结果是不保证排序的，上述 API 查询结果默认是排序的，是历史原因，目前无法简单去掉;现在没有 sql 的 group by，结果就不保证排序

3.当 GroupBy 字段为空时，如 dataset.groupBy(null)，或者 dataSet.groupBy(new String[0]);

dataSet.groupBy(null).sum(a).max(b).finish()等价于没有 groupby。

4. 有聚合函数的 sql: Select sum(a),max(b) from t;如果用 sql，可以简写: Select sum(a),max(b)，不需要 group by 关键字。

groupBy

GroupbyDataSet groupBy(String[] groupFields)

GroupbyDataSet groupBy(String[] groupFields, boolean[] orderByDescs)

返回 GroupbyDataSet 对象

参数	类型	说明
groupFields	String[]	groupBy 字段
orderByDescs	boolean[]	是否 desc 排序

groupBy 示例

```
// groupBy
GroupbyDataSet gds = ds.groupBy(new String[] { "sex" });
gds.sum( s: "age", s1: "totalage"); // 年龄求和
gds.avg( s: "age", s1: "avgage"); // 平局年龄
gds.count( s: "id");// 根据ID计数
DataSet ds1 = gds.finish();
while (ds1.hasNext()) {
    Row r = ds1.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

union

union 操作与 SQL 的 union 类似。两个 DataSet 叠加起来，不支持消除重复。

两个 DataSet 字段个数必须相同，且每个字段的类型必须互相兼容。

字段 Fields 合并规则

默认字段名取左边。

数据类型取左边和右边的兼容类型。

数据类型兼容规则

Number 类型，取最大精度如(Int,Long)取 long，(BigDecimal,Long)取 BigDecimal。

含 Null 类型的，取非 Null 类型。

含 Unknown 类型的，取非 Unknown 类型的。

如果两个 DataSet 字段个数不同，又需要 Union，可以通过 DataSet.addNullField()添加 null 字段，以满足相同个数。

union

public DataSet union(DataSet rightDataSet)

参数	类型	说明
rightDataSet	DataSet	被 union 的 DataSet

```
// union
DataSet ds1 = ds.copy();
DataSet ds2 = ds.union(ds1);
while (ds2.hasNext()) {
    Row r = ds2.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2));
}
```

join

适用于两个 DataSet 的 join，跟 sql 的 join 类似，支持 InnerJoin, LeftJoin, RightJoin, FullJoin, CrossJoin。

方法

JoinDataSet join(DataSet rightDataSet)

JoinDataSet join(DataSet rightDataSet, JoinHint hint)

JoinDataSet join(DataSet rightDataSet, JoinType type)

JoinDataSet join(DataSet rightDataSet, JoinType type, JoinHint hint)

Join 便捷方法

JoinDataSet leftJoin(DataSet rightDataSet)

JoinDataSet leftJoin(DataSet rightDataSet, JoinHint hint)

JoinDataSet rightJoin(DataSet rightDataSet)

JoinDataSet rightJoin(DataSet rightDataSet, JoinHint hint)

JoinDataSet fullJoin(DataSet rightDataSet)

JoinDataSet fullJoin(DataSet rightDataSet, JoinHint hint)

返回 join 后的 JoinDataSet 对象

JoinDataSet 构造器接口

(1)JoinDataSet on(String leftField, String rightField); //类似于 sql 中的 on

left.leftField=right. rightField

(2)JoinDataSet select(String[] leftFields, String[] rightFields);//类似于 sql 中的 select,

分别选择左右表字段

(3)JoinDataSet hint(JoinHint hint);//设置 hint

(4)DataSet finish();//返回 DataSet

参数	类型	说明
rightDataSet	DataSet	dataset
hint	JoinHint	JoinHint 是一些特性指示

		器, 目前只有一个; Join 时, null 值是否当做 0, 为 true 时, 0=null; 为 false 时, 0!=null; 默认值为 false。
type	JoinType	join 类型

```
// join
String algoKey2 = this.getClass().getName();
Algo algo2 = Algo.create(algoKey);
Collection<Object[]> collection2 = new ArrayList<Object[]>();
Object[] obj11 = new Object[] { "1", "语文", 89 };
Object[] obj22 = new Object[] { "1", "数学", 92 };
Object[] obj33 = new Object[] { "1", "英语", 76 };
Object[] obj44 = new Object[] { "2", "语文", 66 };
Object[] obj55 = new Object[] { "3", "数学", 77 };
Object[] obj66 = new Object[] { "4", "英语", 88 };
collection2.add(obj11);
collection2.add(obj22);
collection2.add(obj33);
collection2.add(obj44);
collection2.add(obj55);
collection2.add(obj66);
RowMeta rm2 = new RowMeta(new String[] { "sno", "cname", "score" },
    new DataType[] { DataType.IntegerType, DataType.StringType,
        DataType.IntegerType });
DataSet ds2 = algo2.createDataSet(collection2, rm2);
```

JoinDataSet 构造器

```
// JoinDataSet 构造器
JoinDataSet jds = ds.join(ds2, JoinType.LEFT);
jds.on( s: "id", s1: "sno");
jds.select(new String[] { "id", "name" }, new String[] { "cname", "score" });
DataSet ds3 = jds.finish();
while (ds3.hasNext()) {
    Row r = ds3.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2) + ":" +
        r.get(3));
}
```

leftjoin

```
// leftjoin
JoinDataSet jds = ds.leftJoin(ds2);
jds.on( s: "id", s1: "sno");
jds.select(new String[] { "id", "name" }, new String[] { "cname", "score" });
DataSet ds3 = jds.finish();
DataSet ds4 = ds3.copy();
while (ds3.hasNext()) {
    Row r = ds3.next();
    System.out.println(r.get(0) + ":" + r.get(1) + ":" + r.get(2) + ":" +
        r.get(3));
}
```

count

类似 sql，但不包括 null 值。DataSet 执行 count 后，自动 close。

```
int count(String field, boolean distinct)
```

返回数量。

参数	类型	说明
field	String	计数字段

distinct	boolean	是否去重
----------	---------	------

```
// count  
int num = ds4.count("id", true);  
System.out.println(num);
```

幻

print

打印输出到 console，copy 表示是否 copy dataset。调试时，可以调用 print(true)打印；

跟 ORMUtil.dumpDataSet(dataSet.copy())类似。

void print(boolean copy)

ds3.print(true);

参数	类型	说明
copy	boolean	是否 copy 一个新的

id	name	cname	score
1	唐僧	语文	89
1	唐僧	数学	92
1	唐僧	英语	76
2	悟空	语文	66
3	八戒	数学	77
4	沙僧	英语	88

幻灯片 40

copy

功能描述

copy 是有资源消耗的，默认 DataSet 只能被遍历一次，因为它可能是在线的，即底层可能是 ResultSet，copy 后会变成离线的，占用内存。

什么时候需要 copy

- (1)DataSet 需要遍历多次。
- (2)DataSet 既要被引用，又要遍历使用。
- (3)调试打印时用，ORMUtil.dumpDataSet 或 DataSet.print。

不该 copy 的场景

- (1)dataset.copy().join() copy 结果没有用到。
- (2)调试时用了 copy()，正式代码需要删除掉。

方法

DataSet copy()

返回值

返回一个 copy 的 DataSet。

三、高级 API

cache

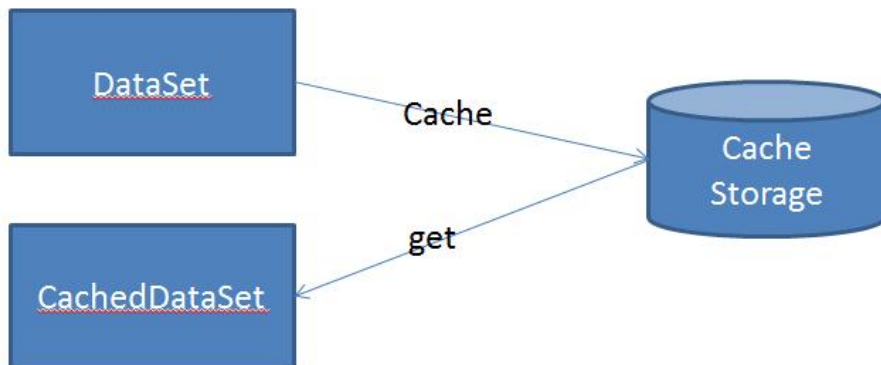
我们可以调用 DataSet 的接口：`public CachedDataSet cache(CacheHint cacheHint)`来

创建一个 CachedDataSet。

dataSet.cache(),dataSet 是离线的, 只能顺序遍历, 遍历一次, 无状态。

CachedDataSet 是将 DataSet 缓存起来, 通过 id 多次访问, 并提供随机按 Range 访问。

CachedDataSet 默认是有超时时间的(30 分钟), 业务可以自己设置。



```
//cache hint
CacheHint hint = new CacheHint(); //缓存指示器
hint.setTimeout(60* 60 * 1000); //默认超时是30分钟
CachedDataSet cachedDataSet = ds.cache(hint);
String id = cachedDataSet.getCacheId();
//获取CachedDataSet
CachedDataSet cachedDataSet2= Algo.getCacheDataSet(id);
Iterator<Row> it = cachedDataSet2.iterator( 0, 5);
while(it.hasNext()) {
    Row row = it.next();
    System.out.println(row.get(0)+" "+row.get(1));
}
//遍历缓存
//Iterator<Row> iterator cachedDataSet.iterator(int begin, int length);
//或者
//List<Row> list = cachedDataSet. getList(int begin, int length);
//关闭
cachedDataSet2.close();
```

Cache Builder

适用于当前 dataset cache 后可以继续添加 row。

//写入缓存

```
CacheHint hint = new CacheHint();
```

```
Hint.setTimeout(60* 60 * 1000); //默认超时是 30 分钟
```

```
CachedDataSet cachedDataSet = DataSet.cache(cacheHint);
```

```
String id = cachedDataSet.getCachedId();
```

//获取 CachedDataSet

```
CachedDataSet cachedDataSet = Algo.get CachedDataSet(id);
```

//遍历缓存

```
Iterator<Row> iterator cachedDataSet.iterator(int begin, int length);
```

或者

```
List<Row> list = cachedDataSet. getList(int begin, int length);
```

//关闭

```
cachedDataSet.close();
```

DataSetBuilder

DataSetBuilder 提供内存构造 DataSet 的功能，DataSetBuilder 不同于

CachedDataSet.Builder。

```
RowMeta rowMeta = xxx;
```

```
DataSetBuilder builder = Algo.createDataSetBuilder(rowMeta);
```

```
builder.append(new Object[]{xxx});
```

```
builder.append(new Object[]{xxx});
```

```
DataSet dataset = builder.build();
```

Input

Input 是一种数据源接口,通过 Input 可以创建一个 DataSet,接口如下:

```
DataSet createDataSet(Input... inputs);
```

执行并行取数, 如果传入多个 Input, 默认启动 4 个并行查询。

Input 有 4 个子类:

OrmInput

DbInput

OqlInput

CollectInput

Input 有一个子接口 CustomizedInput, 供自定义 Input。

```
int n = 8;
OrmInput[] inputs = new OrmInput[n];
for(int i=0;i<n;i++){
    Qfilter[] filters = ...; //把filters拆成多个, 可并行执行
    Inputs[i] = new OrmInput(algoKey, String_entityName, selectFields, filters);
}
DataSet dataset = Algo.createDataSet(inputs);
```

计算表达式

- (1)expr 支持基本四则运算，包括：+,-,*,/,()。
- (2)支持常量数字，字符串。
- (3)支持逻辑表达式： <,>,<=,>=,<>,! =, and, or, xor, not。
- (4)支持宏: NULL, TRUE, FALSE。
- (5)别名: expr alias, expr as alias。
- (6)支持跟 SQL 一样的 Case When 语法，2 种。
- (7)支持函数。

- 1. famount
- 2. famount as amount
- 3. famount * 2 as amount, famount + 2 as amount, famount / fqty as amount, famount * fqty as mount,
- 4. (famount + qty) * 2 as amount
- 5. famount > fqty as b
- 6. famount > fqty and fqty > 100 as c
- 7. null as c, true as c, false as c
- 8. 100 as amount, 'abc' as name
- 9. fname + 'abc' as name

表达式语法

类型	语法	说明
----	----	----

数值四则运算	+, -, *, /, %, ()	%为求余
字符串拼接	+	或者用 concat 函数
字符串 like	like %abc%	跟 sql 类似, 通过%实现左右 like 匹配
别名	expr as alias	A as c, a+b as c
类型转换	Cast(expr as DataType)	
布尔连接符	and, or	
布尔比较符	=, <, <=, >, >=, <>, !=	<,<=,>,>=只适用于数值比 较<> 等价于 !=
Case When 语句	1.Case 表达式 When 值 then 值 [else 值] end2.Case When 布尔表达 式 then 值 [else 值] end	跟 sql 一样, 有两种形式
In	expr in (v1, v2,vN)	跟 sql 一样
Is Null, Is Not Null	1.Expr is Null2.Expr is Not Null	等价于 expr=null, expr<>null
宏	True, False, Null	
参数化	expr>?	?是参数, filter 函数通过 Map 传入值
函数	Func(arg1,arg2)	

日期函数

函数	说明
Now()Date()	返回当前时间
Year(date)	返回 date 年月日中的年
Month(date)	返回 date 年月日中的月
Day(date)	返回 date 年月日中的日
Date(year,month,day)	根据 year, month, day 返回 Date 字段
TO_DATE(str,format)	如 to_date("1999-1-1", " yyyy-MM-dd"), 这里的 format 是 Java 的 DateFormat 规范, 跟 SQL 的不兼容
TO_CHAR(str,format)	如 to_char(date," yyyy-MM-dd"), 这里 的 format 是 Java 的 DateFormat 规范, 跟 SQL 的不兼容
Dateadd(datepart,number,date)	DATEADD 函数需要 3 个参数,格式(参考 SQL): DATEADD(datepart,number,date),datep art 可以为 Year,Month,Day,Hour,Minute,Second
DATEDIF(start_date,end_date,unit)	unit 可以为 Y,M,D

字符串函数

函数	说明
concat(str1,str2,str3)	字符串拼接符，等价于 str1+str2+str3。
contains(left, right)	返回 boolean，left 是否包含 right。
startsWith(left, right)	以什么开头
endsWith(left,right)	以什么结尾
lower(str)	返回小写字符串。
upper(str)	返回大写字符串。
len(str)	返回字符串长度 int。

聚合函数

适用于 GroupBy 的 select 中。

类型	方法	说明	其它
普通聚合函数	sum(String field)sum(String expr, String alias)	expr: 表达式	
普通聚合函数	avg(String field)avg(String expr, String alias)	expr: 表达式	
普通聚合函数	max(String	expr: 表达式	

	field)max(String expr, String alias)		
普通聚合函数	min(String field)min(String expr, String alias)	expr: 表达式	
普通聚合函数	count()count(Strin g alias)	等价于 select count(1)等价于 select(1)	count 不需要指定 field, 因为 count(1)=count(*)
特殊聚合函数	maxP(String field, String propertyField)max P(String expr, String propertyField,Strin g alias)	用 field/expr 计算最 大行, 结果取 propertyField 字段	
特殊聚合函数	minP(String field, String propertyField)min P(String expr, String propertyField, String alias)	用 field/expr 计算最 小行, 结果取 propertyField 字段	

控制	finish()	返回 DataSet 接口	
----	----------	---------------	--

特殊函数

preRowValue()

取上行本列值。

preRowValue("field")

取上行字段 field 的值。

四、注意事项

关闭 DataSet

DataSet 经过多次转换后，会形成一棵引用关系树，如：

```
DataSet c = a.union(b);
```

```
DataSet e = c.select(new String[]{"fid","fname"});
```

```
DataSet f = c.join(d,xxx).finish();
```

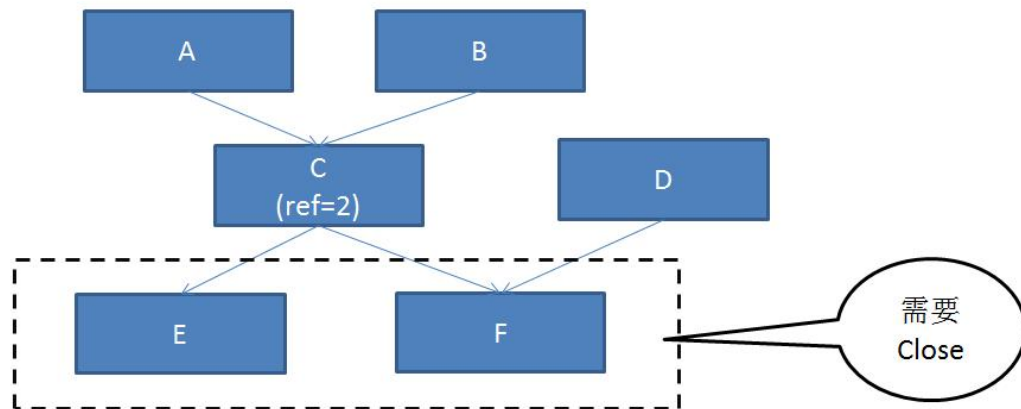
非叶子节点都是被引用的 DataSet。

每个 DataSet 节点都会消耗资源或者引用资源，close 时会释放资源。

DataSet.close 时，会自动 close 引用节点；所以，所有叶子节点 close，会自动 close 整棵树。

叶子节点必须 close。

DataSet 实现 AutoCloseable, 可以使用 jdk7 的 try(resource){}语法, 更简洁的 try{}finally{}。



DataSet 执行 SQL 注意事项

没有 from table。

目前还不支持 join, hashjoin, union, 请使用 api 方式。

除了 join, hashjoin, union, 其他 api 都可以用 sql 查询替代。

group by 和 order by 只支持字段, 还不支持表达式和函数。

Sql 函数不全, 支持的表达式和函数请看后面文档, 具体看文档。

其他注意事项

copy 注意事项

尽量少用 copy, copy 有成本, 会保存数据在磁盘中

Row 访问注意事项

Row 是访问游标, 不是数组, 不能保存起来用

表达式

不能用表达式的地方: orderBy 字段, groupBy 字段, join 中的 on 字段。

表达式中支持哪些函数?

支持的函数主要分三类: 数值函数、字符串函数和日期函数。

ORM 中的 QFilter 是否可以直接在 Algo 中使用?

有很多用户都是将 ORM 中的 QFilter, toString 之后就直接当做过滤条件使用了, 但这样是不对的。QFilter 是 ORM 中的, 与 Algo 中的过滤没有任何关系, 不建议直接 toString 后拿来使用, 因为 QFilter 中的很多特性 Algo 不支持。

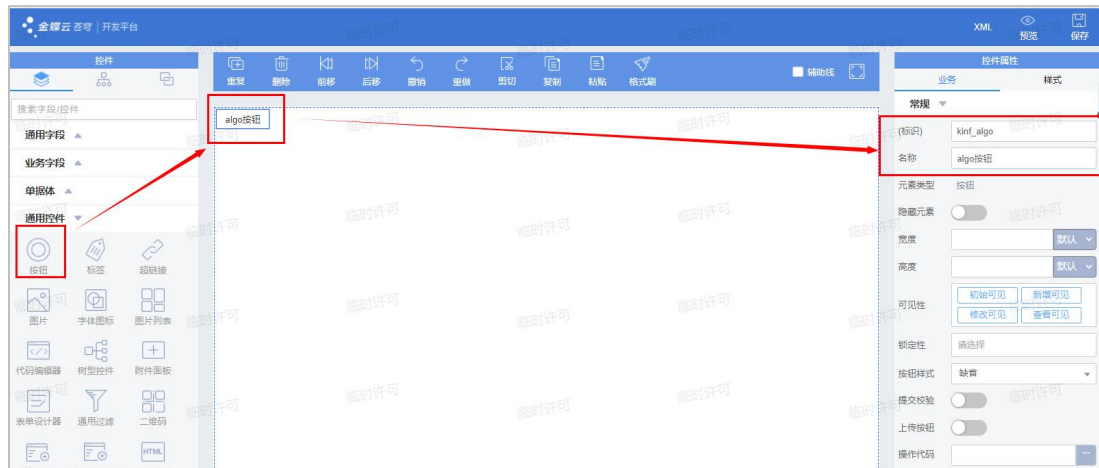
例如, ORM 中的 ftlike、match、join in SQL 等许多操作, Algo 都是不支持的, 并且一些过滤控件绑定的操作符有可能会发生变化, 直接 toString 使用, 与预期很可能不符, 甚至有可能异常。诚然, 他们之间有很多过滤是相通的, 但是不加判断的使用会使程序出现不可预料的错误, 并且 QFilter 的 toString 本来就不是给 algo 来使用的, 所以不建议这样使用, 不可混为一谈。

五、案例演示

创建一个动态表单, 增加按钮控件:



创建一个动态表单，增加按钮控件：

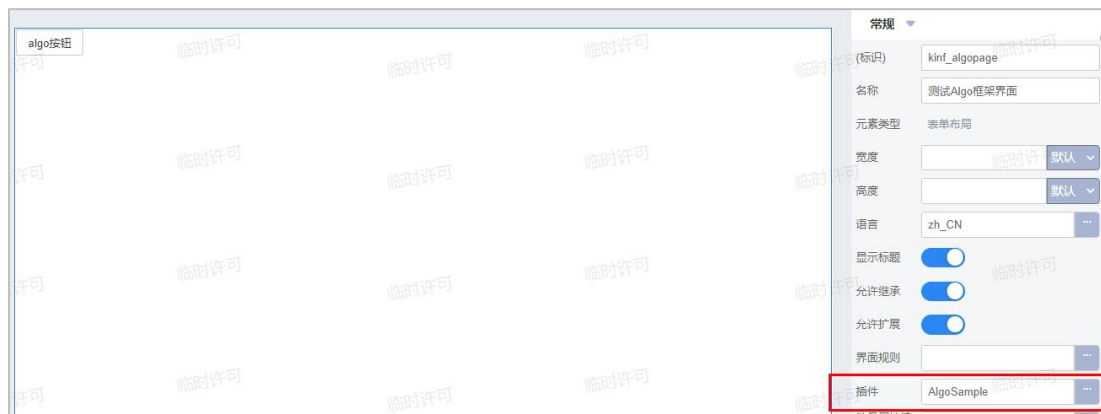


```

public class AlgoSample extends AbstractFormPlugin {
    @Override
    public void registerListener(EventObject e) {
        this.addClickListeners("qyrv_algo");// 注册按钮监听
    }
    @Override
    public void click(EventObject evt) {
        Control control = (Control) evt.getSource();
        String key = control.getKey();
        switch (key) {
            case "qyrv_algo":
                StringBuilder strResult = new StringBuilder();
                // 出差申请单
                String entityName = "qyrv_tripreqbill";
                // 查询字段
                String selectFields =
                    "billNo,qyrv_reqdate,entryentity.qyrv_begindate,entryentity.qyrv_enddate";
                String algoKey = this.getClass().getName();
                //方式 1
                DataSet dSet1 = QueryServiceHelper.queryDataSet(algoKey, entityName, selectFields, null, "");
                while (dSet1.hasNext()) {
                    Row row = dSet1.next();
                    strResult.append(String.format("单据编号: %s;申请日期: %s;开始日期: %s;结束日期: %s;",
                        row.get(0), row.get(1), row.get(2), row.get(3)));
                }
                //方式 2
                String strSql="select A.fbillNo,A.fk_qyrv_reqdate,B.fk_qyrv_begindate,B.fk_qyrv_enddate "
                    + "from tk_qyrv_tripreqbill A "
                    + "inner join tk_qyrv_tripreqbillentry B "
                    + "on A.FID=B.FID";
                DataSet dSet2 = DB.queryDataSet(algoKey, DBRoute.basedata, strSql);
                while (dSet2.hasNext()) {
                    Row row = dSet2.next();
                    //strResult.append(String.format("单据编号: %s;申请日期: %s;开始日期: %s;结束日期: %s;",
                        row.get(0), row.get(1), row.get(2), row.get(3)));
                }
                this.getView().showMessage(strResult.toString());
                break;
            default:
                break;
        }
    }
}

```

点击表单，注册插件



点击预览，点击按钮后，查询出数据弹窗显示。（也可以进行断点调试，查看查询方式，分别使用两种查询方式进行查询）



六、操作实践

需求 1：查询办公用品登记单中的“登记单号”，“登记人”，“登记人联系方式”，“单据状态”，“物品来源渠道”，“物品分类”，“物品”，“计量单位”，“数量”，“单价”，“总金额”。在开发工具后台直接打印查询结果。

需求 2：在需求 1 的基础上，过滤出“单据状态”为已审核的数据。在开发工具后台直接打

印查询结果。

需求 3：在需求 1 的基础上，按照总金额从高到低排序进行显示。在开发工具后台直接打印查询结果。

需求 4：统计需求 1 中查询出来的单据数量（根据单据编号去重后统计）。

需求 5：在需求 1 的基础上，只显示前 5 条记录。

需求 6：统计不同采购人购买物品的总金额。显示采购人，总金额。